ПРОГРАММА РЕАЛИЗУЮЩАЯ ПРОСТУЮ БИНАРНУЮ МУТАЦИЮ

Техническое задание

Листов 7

Руководитель команды	Карузин Иван Михайлович
Технический писатель	Дергалов Никита Олегович
Тестировщик	Каракешишян Дмитрий Мгерович
Разработчик	Вохрамеев Михаил Викторович
Системный аналитик	Шегай Павел Игоревич

Москва 2026

1. ВВЕДЕНИЕ

Настоящее техническое задание устанавливает требования к разработке программы, реализующей простую бинарную мутацию. Программа предназначена для демонстрации изменения одного разряда в двоичной строке и проверки результата на контрольных примерах.

Под бинарной мутацией в рамках данной работы понимается инверсия одного выбранного бита: значение 0 заменяется на 1, а значение 1 заменяется на 0. Изменение остальных разрядов не допускается.

2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ

Основанием для разработки является учебное задание: написать программу, реализующую простую бинарную мутацию, и проверить правильность её работы на пяти примерах.

Наименование разработки — «Программа, реализующая простую бинарную мутацию». Краткое обозначение проекта — BinaryMutation.

3. НАЗНАЧЕНИЕ РАЗРАБОТКИ

Программа предназначена для применения в учебных целях при изучении генетических алгоритмов, представления бинарных хромосом и операторов мутации. Результат работы программы должен быть понятен пользователю и пригоден для проверки вручную.

4. ТРЕБОВАНИЯ К ПРОГРАММЕ

4.1. Функциональные требования.

— программа должна принимать на вход двоичную строку, состоящую только из символов 0 и 1;

— программа должна выбирать одну позицию для мутации случайным образом из диапазона от 1 до длины строки;

— программа должна инвертировать бит в выбранной позиции и сохранять остальные биты без изменений;

— программа должна выводить исходную строку, номер мутированного разряда и строку после мутации;

— для контрольной проверки программа должна поддерживать передачу фиксированной позиции мутации либо иной воспроизводимый способ её задания.

4.2. Требования к входным данным.

Длина входной строки должна быть не менее одного символа. При вводе пустой строки, символов, отличных от 0 и 1, либо номера позиции вне допустимого диапазона программа должна вывести понятное сообщение об ошибке и не выполнять мутацию.

4.3. Требования к результату.

Длина результирующей строки должна совпадать с длиной исходной строки. Ровно один символ в результирующей строке должен отличаться от соответствующего символа исходной строки, причём этот символ должен находиться в выбранной позиции.

5. СОСТАВ И АЛГОРИТМ РАБОТЫ ПРОГРАММЫ

Программа выполняется в консольном режиме. Реализация допускается на любом языке программирования, поддерживающем работу со строками и генерацию случайных чисел; рекомендуемая среда выполнения — Python 3.10 или выше.

Алгоритм работы программы включает следующие действия:

— получение и проверка двоичной строки;

— выбор позиции мутации случайно или получение фиксированной позиции для тестового запуска;

— инверсия бита в выбранной позиции;

— формирование и вывод результата.

6. ПЛАН ИСПЫТАНИЙ ПО IEEE 829

6.1. Идентификатор плана испытаний.

Идентификатор тест-плана — TP-BM-001. План подготовлен с учётом структуры стандарта IEEE 829 и применяется к версии программы BinaryMutation, представляемой на приёмку.

6.2. Объект и цель испытаний.

Объектом испытаний является консольная программа, выполняющая инверсию одного бита двоичной строки. Цель испытаний — подтвердить, что программа корректно выбирает или принимает позицию мутации, меняет ровно один бит и возвращает ожидаемый результат.

6.3. Элементы и функции, подлежащие испытаниям.

— проверка допустимости входной двоичной строки;

— обработка фиксированной позиции мутации в контрольном режиме;

— инверсия значения 0 в 1 и значения 1 в 0;

— сохранение длины строки и неизменности остальных разрядов;

— вывод исходных данных, позиции и результата.

6.4. Функции, не подлежащие испытаниям.

В рамки данного тест-плана не входят нагрузочное тестирование, оценка производительности на больших объёмах данных, тестирование графического интерфейса и проверка интеграции с внешними системами, поскольку они не предусмотрены постановкой задачи.

6.5. Подход к испытаниям.

Применяется функциональное тестирование методом чёрного ящика. Для воспроизводимости позиции мутации в каждом контрольном запуске задаются явно. Тестировщик сравнивает фактическую строку с ожидаемой, а также проверяет количество изменённых позиций.

6.6. Критерии прохождения и отказа.

Тест считается пройденным, если фактический результат полностью совпадает с ожидаемым, длина строки не изменена и различается ровно один разряд. Тест

считается непройденным при расхождении результата, изменении нескольких разрядов, некорректной обработке входа или аварийном завершении программы.

6.7. Критерии приостановки и возобновления.

Испытания приостанавливаются при невозможности запуска программы, отсутствии согласованной среды выполнения либо обнаружении ошибки, препятствующей выполнению следующего теста. Испытания возобновляются после устранения причины, сборки исправленной версии и повторной проверки ранее непройденных тестов.

6.8. Контрольные примеры.

Позиции нумеруются слева направо, начиная с единицы. В каждом запуске фиксированная позиция передаётся программе как параметр тестового режима.

Таблица 1 — Контрольные примеры бинарной мутации

№	Исходная строка	Позиция мутации	Ожидаемый результат
1	00000	2	01000
2	11111	3	11011
3	101010	6	101011
4	0	1	1
5	1001	2	1101

6.9. Тестовые артефакты и ответственность.

Таблица 2 — Тестовые артефакты

Артефакт	Содержание	Ответственный
Техническое задание	Требования к программе и план испытаний	Разработчик
Контрольные данные	Пять строк, позиции и ожидаемые результаты	Тестировщик
Протокол испытаний	Фактические результаты, статус и дата запуска	Тестировщик
Исходный код	Реализация мутации и обработки ошибок	Разработчик

6.10. Среда, задачи, риски и утверждение.

Испытания выполняются на компьютере с установленной средой запуска выбранного языка программирования; для рекомендуемой реализации используется Python 3.10 или выше. Задачи тестировщика: подготовить контрольные данные, выполнить пять запусков, сравнить результаты и оформить протокол. Основной риск — случайный выбор позиции сделает результаты невоспроизводимыми; он

устраняется фиксированной позицией или фиксированным начальным значением генератора. План считается утверждённым после согласования разработчиком и проверяющим лицом.

7. КРИТЕРИИ ПРИЁМКИ

Программа считается принятой, если она запускается без ошибок в согласованной среде выполнения, корректно обрабатывает допустимые входные данные, сообщает об ошибочных данных и успешно проходит все пять контрольных примеров.

Приёмка выполняется путём демонстрации исходной строки, выбранной позиции и результата мутации для каждого примера. Расхождение хотя бы с одним ожидаемым результатом является основанием для доработки программы.

8. СРОКИ РАЗРАБОТКИ

Работы выполняются последовательно в течение семи рабочих дней. Продолжительность этапов и дни выполнения приведены в таблице 3; суммарная длительность не превышает семи дней.

Таблица 3 — Этапы разработки и сроки выполнения

№	Этап	Содержание работ	Длительность, дней	Дни выполнения
1	Анализ и планирование	Уточнение требований, подготовка тест-плана и контрольных данных	1	1
2	Реализация	Разработка обработки ввода, выбора позиции и инверсии бита	2	2–3
3	Испытания и доработка	Выполнение пяти тестов, анализ результатов и исправление выявленных ошибок	2	4–5
4	Документация и приёмка	Подготовка инструкции и протокола, демонстрация результата	2	6–7
	Итого		**7**	**1–7**

9. СОСТАВ ДОКУМЕНТАЦИИ

В состав результатов разработки входят исходный код программы, инструкция по запуску и краткий протокол испытаний с результатами пяти контрольных

примеров. Исходный код должен быть снабжён комментариями, поясняющими обработку ввода и операцию инверсии бита.